

Embedded

FA2025 • 2026-09-21

Embedded 102: Microcontroller Programming

Krishnan

Krishnan Shankar

- SIGPwny Admin
- Computer Engineering '27
- Fun fact: I built my own bike



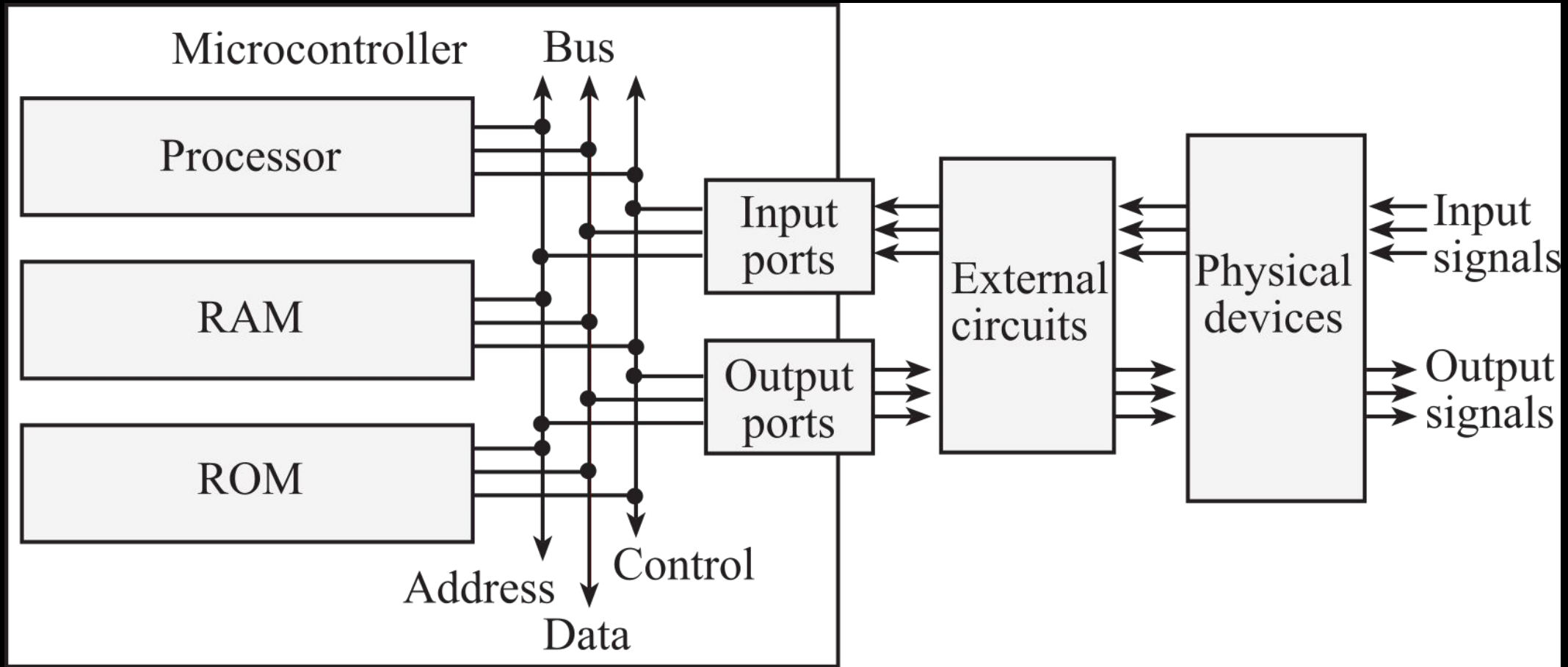
Meeting content can be found at
sigpwny.com/meetings.



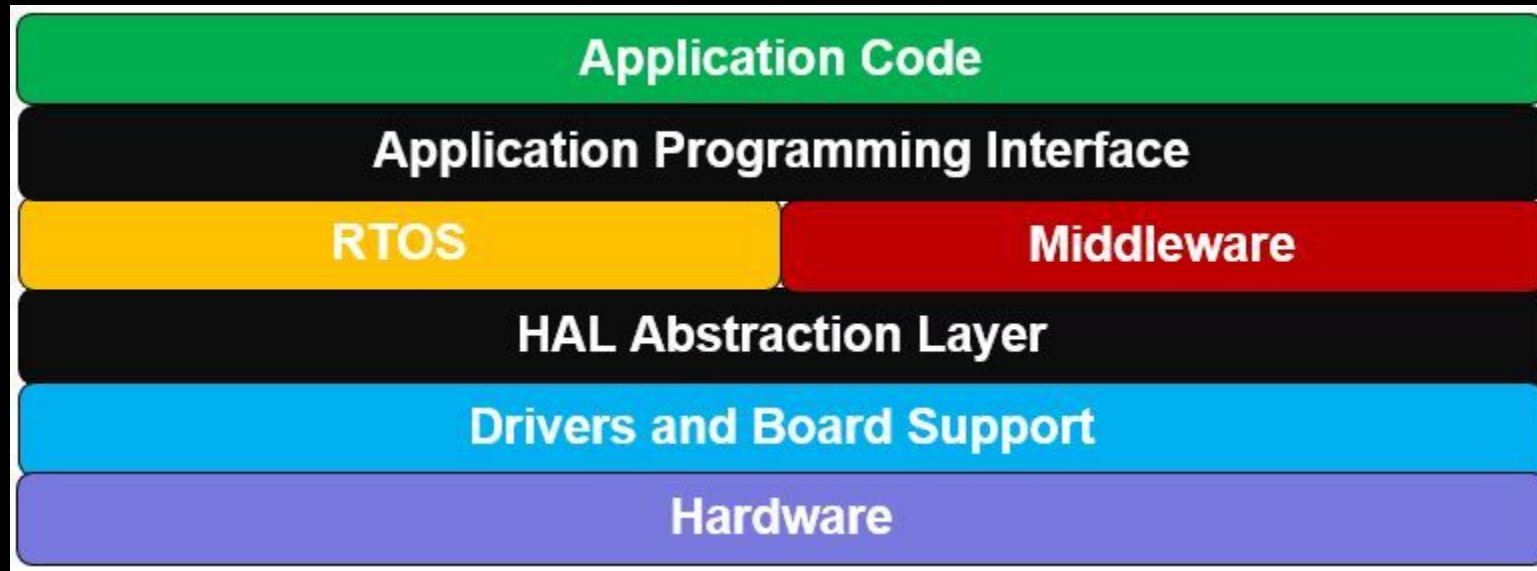
The Basics



Recall: Embedded Systems



Embedded Software Stack



1. The C programming language

- Compiled “high-level” programming language
- Created in the 70s by Dennis Ritchie while developing Unix
- Widely used for low-level programming
- Offers easy access to memory and hardware peripherals

```
#include "stdio.h"
int main() {
    int x = 1;
    int y = 2;
    int z = x + y;
    printf("z = %d", z);
    return 0;
}
```



2. Compilation

- The C program gets turned into assembly code
- Which assembly language?
 - You can pick!
 - e.g., you can compile your code “for” x86_64 (i386), ARM (aarch64), etc.



2. Compilation

```
#include "stdio.h"

int main() {
    int x = 1;
    int y = 2;
    int z = x + y;
    printf("z = %d", z);
    return 0;
}
```

```
1  main:
2      push    {r11, lr}
3      mov     r11, sp
4      sub     sp, sp, #24
5      mov     r0, #0
6      str     r0, [sp, #4]
7      str     r0, [r11, #-4]
8      mov     r0, #1
9      str     r0, [r11, #-8]
10     mov     r0, #2
11     str     r0, [sp, #12]
12     ldr     r0, [r11, #-8]
13     ldr     r1, [sp, #12]
14     add     r0, r0, r1
15     str     r0, [sp, #8]
16     ldr     r1, [sp, #8]
17     ldr     r0, .LCPI0_0
18 .LPC0_0:
19     add     r0, pc, r0
20     bl      printf
21     ldr     r0, [sp, #4]
22     mov     sp, r11
23     pop     {r11, lr}
24     bx      lr
25 .LCPI0_0:
```



3. Bitstream

- Based on the ISA (instruction set architecture), every assembly instruction “becomes” a specific binary string
- This, along with some other things, forms an object file
- This object file, along with some other things, forms a bitstream
- This bitstream gets sent to the MCU over USB (using UART)

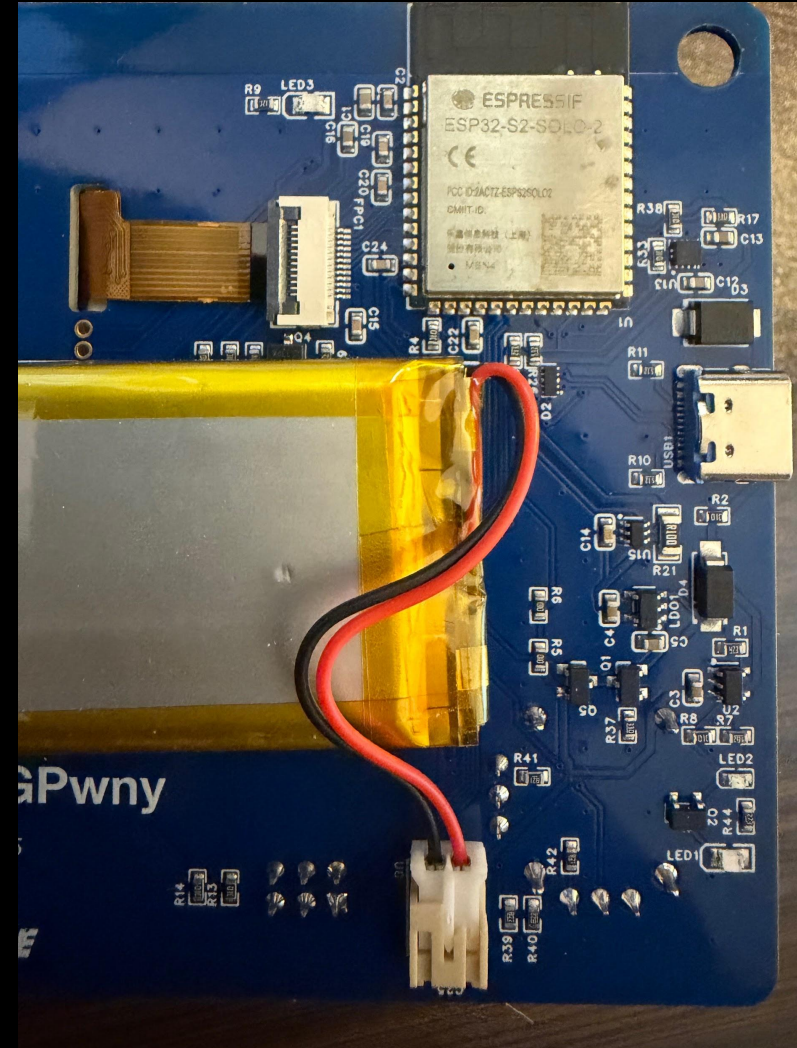


In Practice



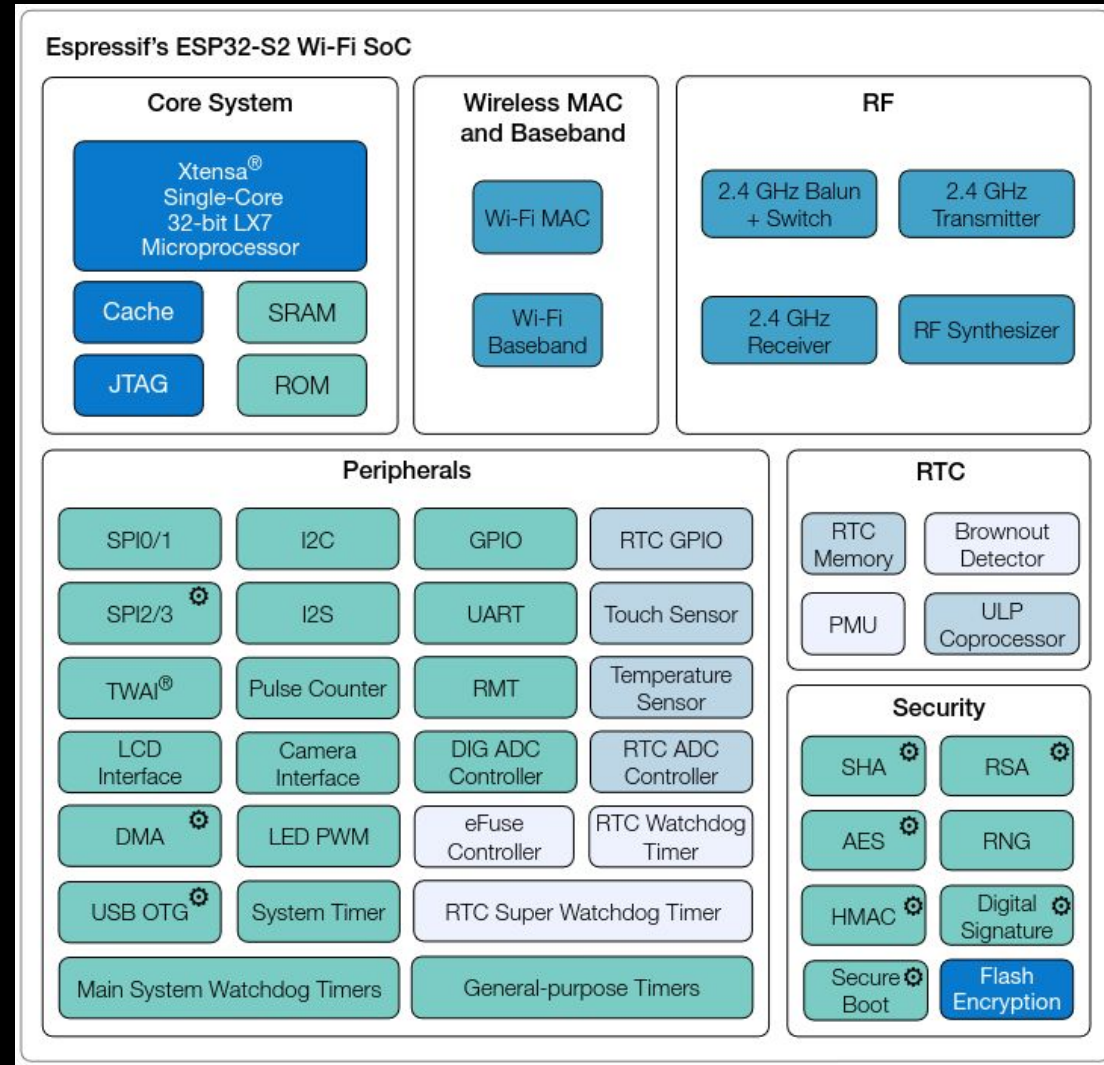
Example: FallCTF Badges

- ESP32-S2-SOLO-2
- [Datasheet](#)
- [User Guide](#)
- [ISA Spec](#)



Example: FallCTF Badges

- ESP32-S2-SOLO-2
- [Datasheet](#)
- [User Guide](#)
- [ISA Spec](#)



Example: FallCTF Badges

- ESP32-S2-SOLO-2
- [Datasheet](#)
- [User Guide](#)
- [ISA Spec](#)

4.1.1.1 CPU

ESP32-S2 contains one low-power Xtensa® 32-bit LX7 microprocessor with the following features:

- 7-stage pipeline that supports the clock frequency of up to 240 MHz
- 16/24-bit Instruction Set providing high code-density
- support for 32-bit multiplier and 32-bit divider
- unbuffered GPIO instructions
- support for 32 interrupts at six levels
- support for windowed ABI with 64 physical general registers
- support for trace function with TRAX compressor, up to 16 KB trace memory
- JTAG for debugging

For information about the Xtensa® Instruction Set Architecture, please refer to

[Xtensa® Instruction Set Architecture \(ISA\) Summary](#)



Example: FallCTF Badges

- ESP32-S2-SOLO-2
- [Datasheet](#)
- [User Guide](#)
- [ISA Spec](#)

8.3.20 AND—Bitwise Logical And

Instruction Word (RRR)

23	20	19	16	15	12	11	8	7	4	3	0
0	0	0	1	0	0	0	0	r	s	t	0
4				4				4	4	4	4

Required Configuration Option

Core Architecture (See [Core Architecture](#) on page 77)

Assembler Syntax

```
AND ar, as, at
```

Description

AND calculates the bitwise logical and of address registers `as` and `at`. The result is written to address register `ar`.

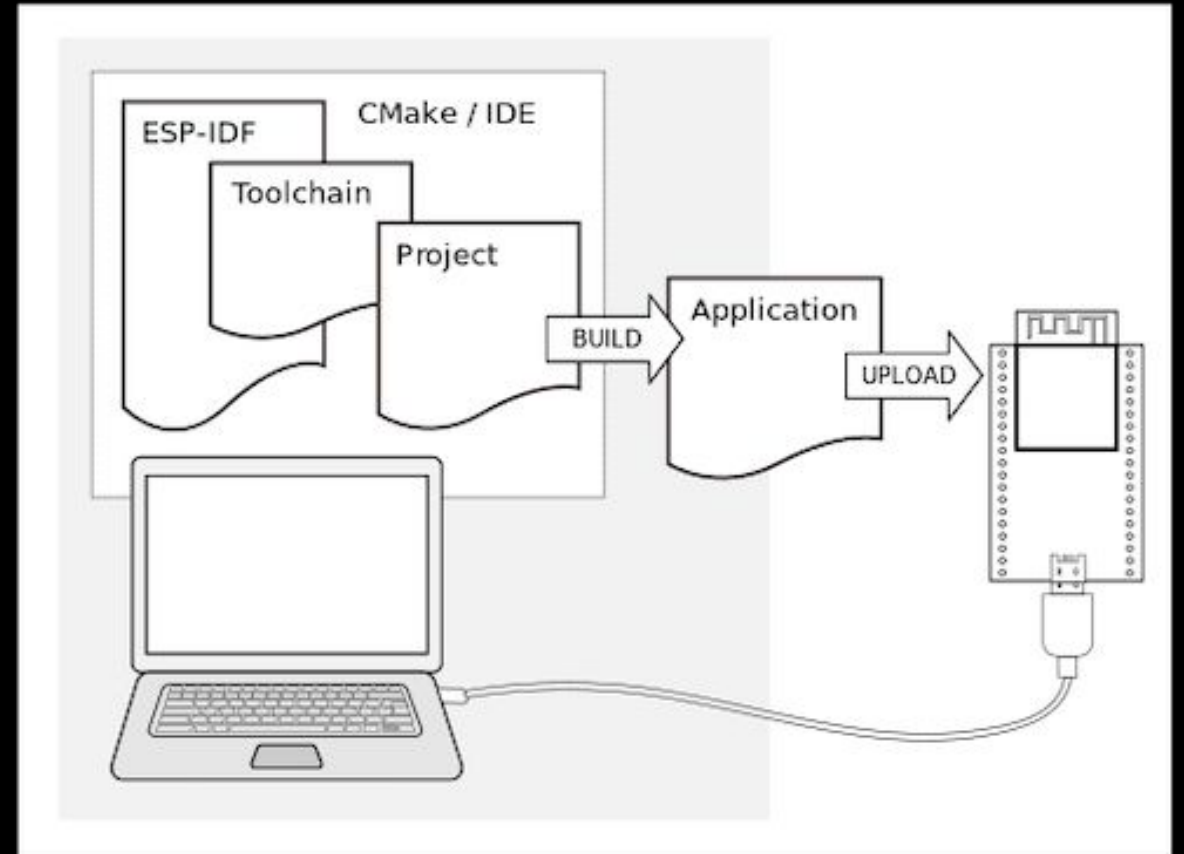
Operation

```
AR[r] ← AR[s] and AR[t]
```



The Overview

- We need a **toolchain** to compile our C code (`xtensa-esp-elf-*`)
- We need a **build tool** to create an ESP32-compatible application (`CMake`, `Ninja`)
- We need a **flashing tool** to copy that application to Flash Memory (`esptool.py`)

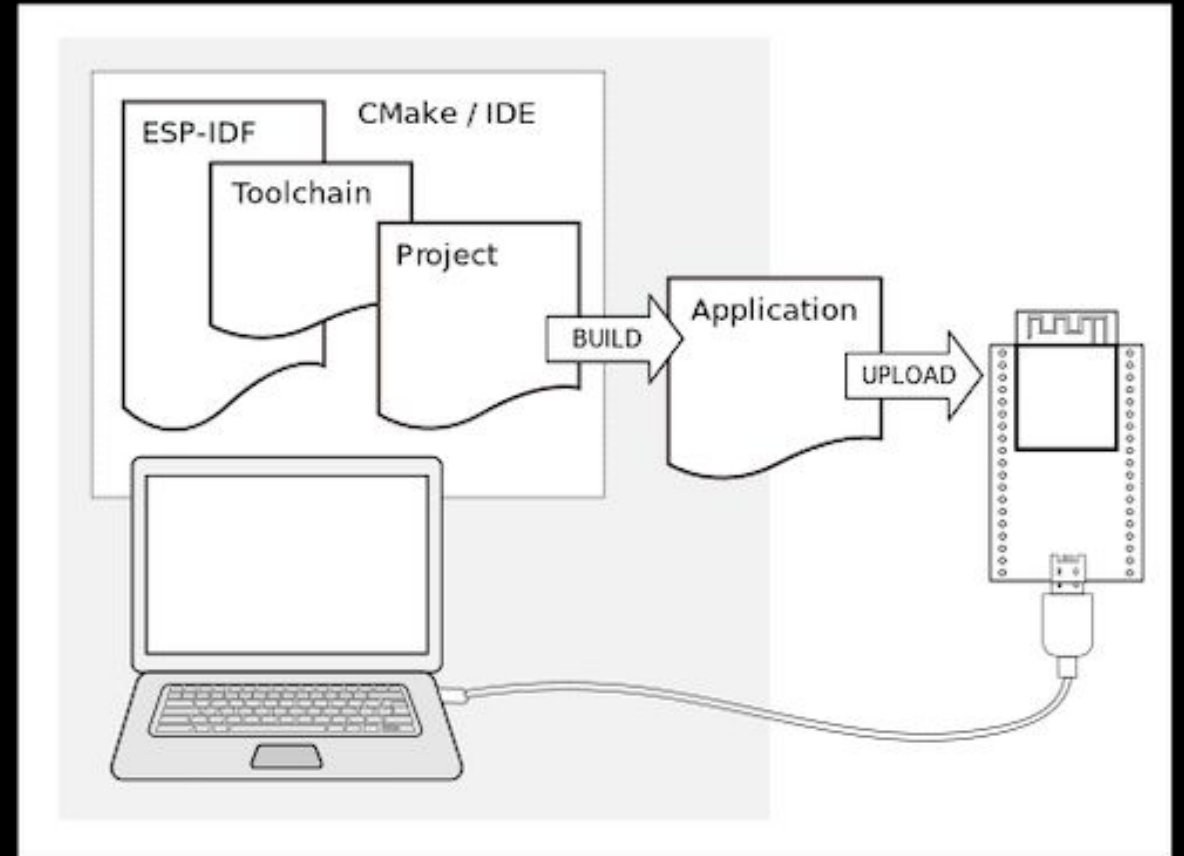


Source: [ESP-IDF Programming Guide](#)



The Overview

- We need a **toolchain** to compile our C code (`xtensa-esp-elf-*`)
- We need a **build tool** to create an ESP32-compatible application (`CMake`, `Ninja`)
- We need a **flashing tool** to copy that application to Flash Memory (`esptool.py`)
- **Espressif's IDF handles all of these!**



Source: [ESP-IDF Programming Guide](#)



Installing ESP IDF

- **E**sspressif **I**oT **D**evelopment **F**ramework
- <https://docs.espressif.com/projects/esp-idf/en/stable/esp32s2/get-started/index.html>
- Simplest method:
 - Install the “ESP-IDF” VSCode Extension
 - Find the ESP-IDF tab on the left
 - Click “Configure ESP-IDF Extension”
 - Use “GitHub” download server
 - Click “Install”



What does it look like?

- `cd project-directory`
- `idf.py set-target esp32s2`
- `idf.py menuconfig`
 - Component config > ESP System Settings > Channel for console output
 - Change from “Default: UART0” to “USB CDC”
- Switch the ESP to bootloader mode (aka download mode)
 - Press and hold “B” on the front (boot button)
 - Press and release “SW4” on the back (reset button)
 - Release “B”
- `idf.py -p PORT -b 115200 flash`
- `idf.py -p PORT -b 115200 monitor`



What does it look like?

- We will do this for real next Monday, this is just a sneak peek



What does it look like?

- `cd project-directory`
- `idf.py set-target esp32s2`
- `idf.py menuconfig`
 - Component config > ESP System Settings > Channel for console output
 - Change from “Default: UART0” to “USB CDC”
- Switch the ESP to bootloader mode (aka download mode)
 - Press and hold “B” on the front (boot button)
 - Press and release “SW4” on the back (reset button)
 - Release “B”
- `idf.py -p PORT -b 115200 flash`
- `idf.py -p PORT -b 115200 monitor`



What does it look like?

- `cd project-directory`
- `make`
- `esptool.py -c esp32s2 elf2image --flash_mode="dio" --flash_freq "40m" --flash_size "4MB" -o main.bin main.elf`
- Switch the ESP to bootloader mode
- `esptool.py -c esp32s2 -p PORT -b 115200 --before default_reset -a hard_reset write_flash -z --flash_mode dio --flash_freq 40m --flash_size detect 0x1000 main.bin`



Deep Dive

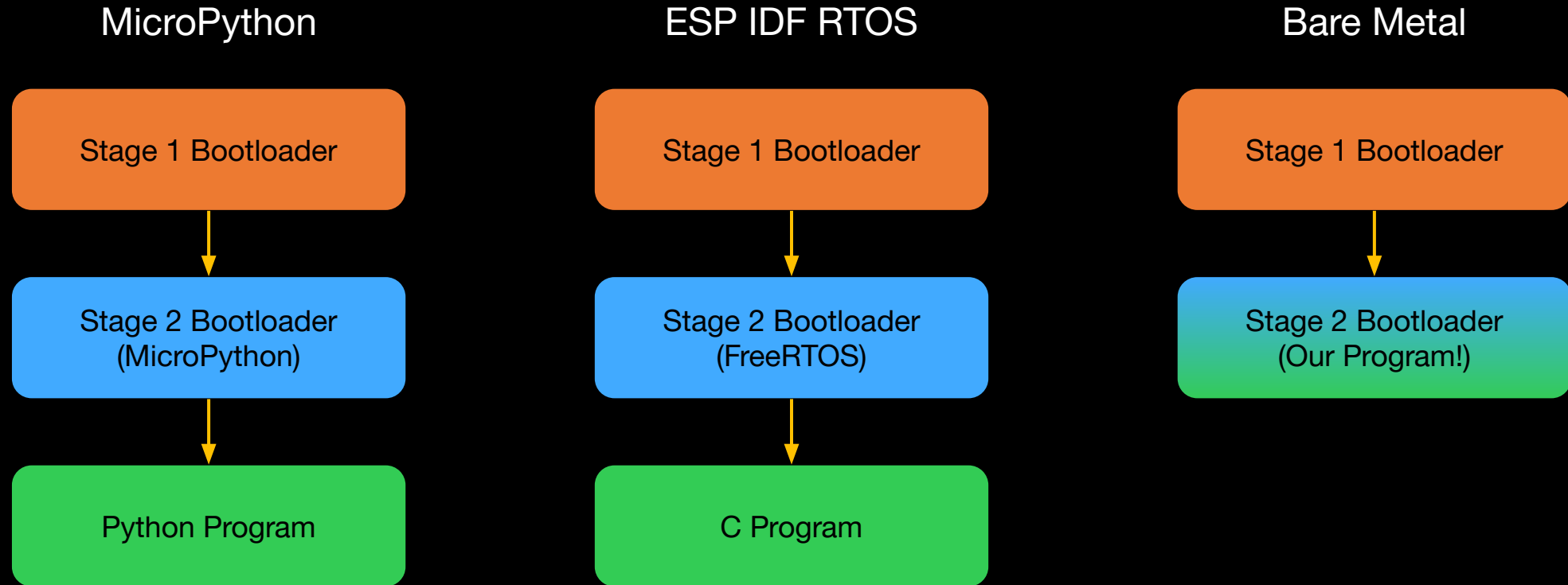


Real-Time Operating System

- The most common form of “operating system” found on embedded devices
- Helps with a few important things:
 - Interrupts
 - Context switching
 - Heaps/Allocator
- But is generally not a full-fledged operating system (filesystem, virtual memory, etc.)



Boot Process



The Boot Process, in words

- Stage 1 Bootloader
 - Hardcoded, cannot be modified
 - Once finished, it jumps to a fixed memory address (`0x1000`)
- Stage 2 Bootloader
 - Is flashed to address `0x1000`
 - Can be anything we want, but is generally FreeRTOS

Flashing

Then deploy the firmware to the board, starting at address 0x1000:

```
esptool.py --baud 460800 write_flash 0x1000 ESP32_BOARD_NAME-DATE-VERSION.bin
```

Replace `ESP32_BOARD_NAME-DATE-VERSION.bin` with the `.bin` file downloaded from this page.

Source: [MicroPython \(ESP32 S2\)](#)



The Boot Process, in words

- Stage 1 Bootloader
 - Hardcoded, cannot be modified
 - Once finished, it jumps to a fixed memory address (0x1000)
- Stage 2 Bootloader
 - Is flashed to address 0x1000
 - Can be anything we want, but is generally FreeRTOS
- FreeRTOS
 - Reads address 0x8000 to view a list of all flashed images
 - Jumps to address 0x10000 to run the first user program
- The User Program
 - Is flashed to address 0x10000



So How Do We Run Bare Metal?

- Write a C Program
 - To run bare metal, it should have a `call_start_cpu0` function
- Write a custom linker script
 - This tells the compiler where to put certain parts of main.c
 - For example, code in IRAM and variables in DRAM
- Write a Makefile
 - This will compile your code to `main.elf` using the linker script
- Convert the ELF file to a binary image using esptool
- Flash the binary image at 0x1000 using esptool



The Bare Metal C Program

```
#include <string.h>

extern unsigned int _sbss, _ebss, _sidata, _sdata, _edata;

void __attribute__((noreturn)) call_start_cpu0() {
    memset(&_sbss, 0, (&_ebss - &_sbss) * sizeof(_sbss));
    memmove(&_sdata, &_sidata, (&_edata - &_sdata) * sizeof(_sdata));

    main();
}

static volatile int a = 0;

int main(void) {
    while (1) {
        ++a;
    }
    return 0;
}
```

Source: [Vivonomicon](https://vivanomicon.com)



Next Meetings

2026-09-28 • Next Monday

- Embedded 102 Part 2: Microcontroller Programming Lab



Meeting content can be found at
sigpwny.com/meetings.

